

Exception Handling

Q. What is an exception?

An exception is a runtime event which occurs at program runtime and it is responsible for disturbing the normal flow of applications called exceptions.

If we want to prevent runtime program disturbance or flow disturbance or abnormal execution at runtime java suggest us use Exception handling technique

Q What is exception handling?

Exception handling is technique which help us to avoid exception at runtime or avoid abnormal termination of code or program at runtime

Q. What is the benefit of exception handling or Why use exception handling?

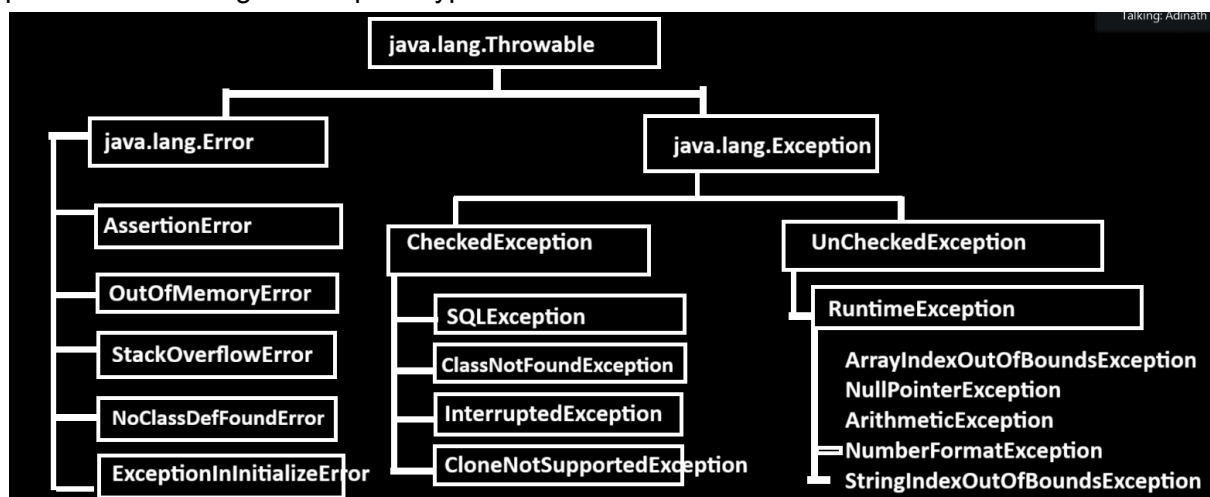
1. It help programmer to detect runtime error
2. Skip the code in which exception may be occur and execute remaining in safe zone
3. Developer can create customize exceptions according to project requirement

Types of exception

There are three types of exception in java

1. **Checked exception** : checked exception means exception occur at program compile time called as checked exceptions
2. **Unchecked exception** : unchecked exception means exception occur at program runtime called as unchecked exceptions
3. **Error** : error is a part of exception but not handled by a programmer called as error. Error is not handle by the developer

If we want to work with this type of exception using java we have some inbuilt classes provided according to exception types.



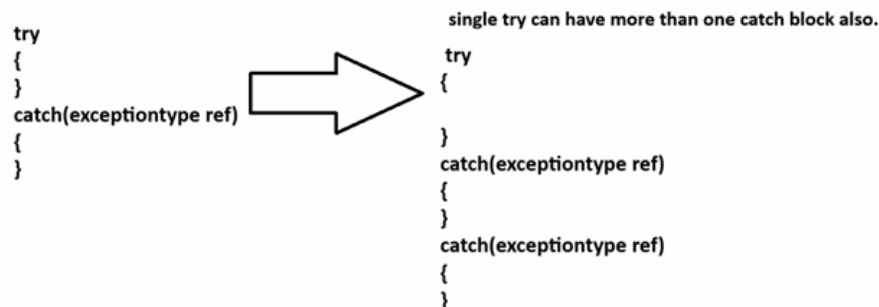
If we want to handle the exception in Java we have five types of keyword

1. Try
2. Catch
3. Finally
4. Throw
5. Throws

try: try block is used for write code in which exception may be occur and when exception occur in try block then JVM create one exception object and hand over to catch for further execution. Means exception generate in try block then JVM create one exception object and hand over to catch and handle it

catch: catch block hold exception object thrown by try block or send by try block and handle means normally catch block is used for execute logic after exception occur.

How to write try and catch practically



Exceptions

ArithmeticException : normally this class provided by java to manage some arithmetic exception problems like as divide by zero.

```

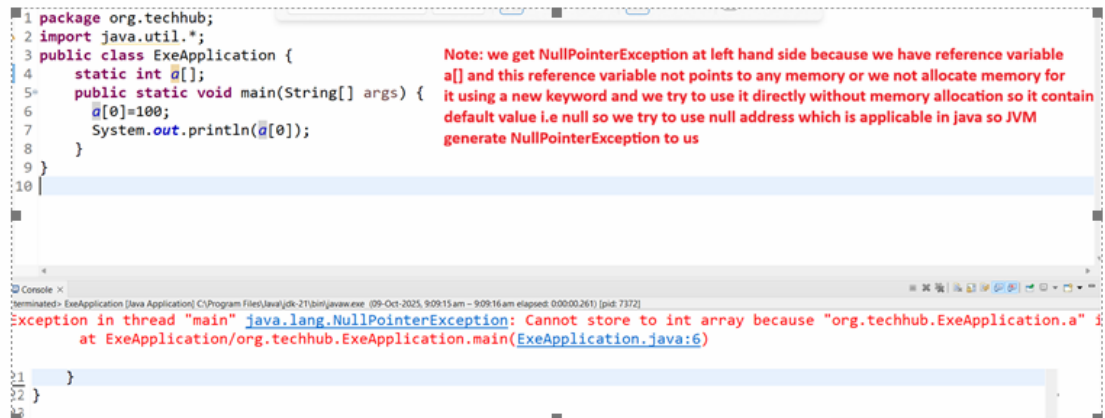
1 package org.techhub;
2 import java.util.*;
3 public class ExeApplication {
4     public static void main(String[] args) {
5         Scanner xyz = new Scanner(System.in);
6         int a,b,c;
7         System.out.println("Enter two values");
8         a=xyz.nextInt(); //10
9         b=xyz.nextInt(); //0
10        try {
11            c=a/b; //10/0 //new ArithmeticException();
12            System.out.printf("Division is %d\n",c);
13        }
14        catch(ArithmeticException ex) {
15            System.out.println("Error is "+ex);
16        }
17        System.out.println("Logic1");
18        System.out.println("Logic2");
19        System.out.println("Logic3");
20    }
21 }

```



NullPointerException: NullPointerException normally occur when we use any reference without memory allocation or without new keyword then we get NullPointerException

Example: if we use array without new keyword or if call any class method using reference without new keyword then there is possibility we get NullPointerException

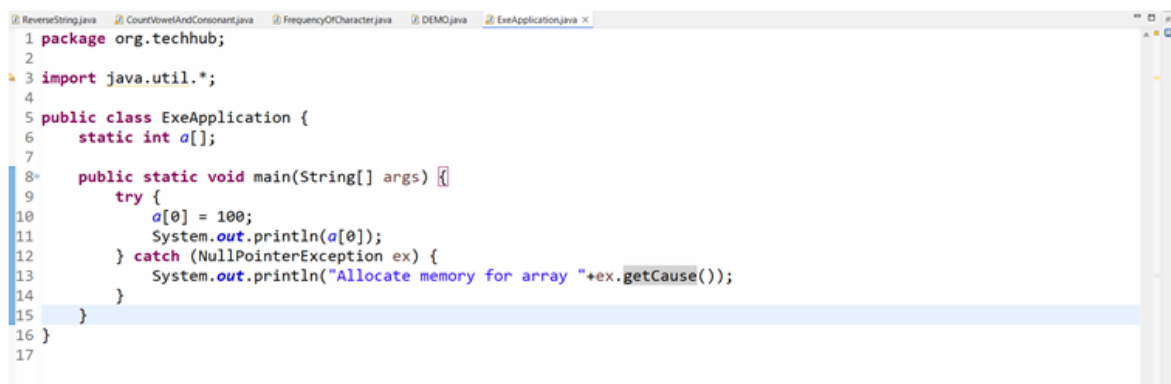


```
1 package org.techhub;
2 import java.util.*;
3 public class ExeApplication {
4     static int a[];
5     public static void main(String[] args) {
6         a[0]=100;
7         System.out.println(a[0]);
8     }
9 }
10
```

Note: we get NullPointerException at left hand side because we have reference variable a[] and this reference variable not points to any memory or we not allocate memory for it using a new keyword and we try to use it directly without memory allocation so it contain default value i.e null so we try to use null address which is applicable in java so JVM generate NullPointerException to us

Exception in thread "main" java.lang.NullPointerException: Cannot store to int array because "org.techhub.ExeApplication.a" is null at ExeApplication/org.techhub.ExeApplication.main(ExeApplication.java:6)

If we want to solve problem at runtime or manage exception at runtime we have handle the NullPointerException using catch block



```
1 package org.techhub;
2
3 import java.util.*;
4
5 public class ExeApplication {
6     static int a[];
7
8     public static void main(String[] args) {
9         try {
10             a[0] = 100;
11             System.out.println(a[0]);
12         } catch (NullPointerException ex) {
13             System.out.println("Allocate memory for array "+ex.getCause());
14         }
15     }
16 }
17
```

Example of NullPointerException

```
package org.techhub;
```

```
import java.util.*;
```

```
class ABC
```

```
{
```

```
    void show() {
```

```
        System.out.println("Hey I am show");
    }
}

public class ExeApplication {

    static ABC a1;

    public static void main(String[] args) {

        try {
            a1.show();
        }

        catch(Exception ex) {

            System.out.println("Error is "+ex);

        }

    }

}
```

ArrayIndexOutOfBoundsException : this exception occur when we try to access element using index more than size or equal of array

```
1 package org.techhub;
2
3 import java.util.*;
4
5 public class ExeApplication {
6
7     public static void main(String[] args) {
8         int a[]=new int[2]; //0 1
9         a[2]=100;
10        System.out.println(a[2]);
11    }
12 }
13
```

Note: if we think about array we have array with size 2 means it contain two index 0 & 1 but we store element on index 2nd which is not present in array so this is major reason we get exception at runtime name as ArrayIndexOutOfBoundsException

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 2 out of bounds for length 2
at ExeApplication/org.techhub.ExeApplication.main(ExeApplication.java:9)

So we can manage this exception by using ArrayIndexOutOfBoundsException class in catch block.

```
1 package org.techhub;
2
3 import java.util.*;
4
5 public class ExeApplication {
6
7     public static void main(String[] args) {
8         try {
9             int a[]=new int[2]; //0 1
10            a[2]=100;
11            System.out.println(a[2]);
12        }
13        catch(ArrayIndexOutOfBoundsException ex) {
14            System.out.println("Error is "+ex);
15        }
16    }
17 }
18
```

Error is java.lang.ArrayIndexOutOfBoundsException: Index 2 out of bounds for length 2

NegativeArraySizeException:

```
3 import java.util.*;
4
5 public class ExeApplication {
6
7     public static void main(String[] args) {
8         try {
9             int a[] = new int[-2]; // 0 1
10            a[2] = 100;
11            System.out.println(a[2]);
12        }
13        catch (ArrayIndexOutOfBoundsException ex) {
14            System.out.println("Error is " + ex);
15        }
16    }
17 }
```

Note: if we think about left hand side code we get exception at program runtime name as `NegativeArraySizeException` because when we create array then we allocate memory of array but we provide size of array negative which is not allowed so we get exception
But if we want to handle this exception java provide one inbuilt class to us known as `NegativeArraySizeException`

Exception in thread "main" java.lang.NegativeArraySizeException: -2
at ExeApplication/org.techhub.ExeApplication.main(ExeApplication.java:9)

Handle above exception by writing extra catch with `NegativeArraySizeException` class.

```
1 package org.techhub;
2
3 import java.util.*;
4 public class ExeApplication {
5     public static void main(String[] args) {
6         try {
7             int a[] = new int[-2]; // 0 1
8             a[2] = 100;
9             System.out.println(a[2]);
10        } catch (ArrayIndexOutOfBoundsException ex) {
11            System.out.println("Error is " + ex);
12        } catch (NegativeArraySizeException ex) {
13            System.out.println("Array size must be positive " + ex.getMessage());
14        }
15    }
16 }
17 }
```

Array size must be positive -2

NumberFormatException: `NumberFormatException` class normally occur when we try to perform conversion between string to other type.

Example: suppose we have String str="123" and we want to convert it in integer so we statement `int a = Integer.parseInt(str);` but we have String str="123a" or String str="123 " if we think this type of string it contain alphabet or may be space or other non numeric value and we try to convert in integer or any other number type of value so this conversion is not possible so JVM generate the exception to us known as `NumberFormatException`

Examples

IllegalArgumentException: this type of exception occur when we pass wrong parameter to method or constructor then we can use `IllegalArgumentException`

Example: Suppose if we think about Vector and we have constructor Vector(int size) so we cannot pass negative values in Vector and if we try to give the negative size to Vector then we get exception at runtime name as IllegalArgumentException

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Vector v = new Vector(-2);
        System.out.println(v.capacity());
    }
}

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP
Exception in thread "main" java.lang.IllegalArgumentException: Illegal Capacity: -2
    at java.base/java.util.Vector.<init>(Vector.java:144)
    at java.base/java.util.Vector.<init>(Vector.java:159)
    at VAPP.main(VAPP.java:5)

C:\Program Files\Java\jdk-24\bin>
```

NoSuchElementException: this exception occurs when we try to access element but element does not exist then NoSuchElementException occurs.

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner("");
        int val=xyz.nextInt();
    }
}

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP
Exception in thread "main" java.util.NoSuchElementException
    at java.base/java.util.Scanner.throwFor(Scanner.java:962)
    at java.base/java.util.Scanner.next(Scanner.java:1619)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2284)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2238)
    at VAPP.main(VAPP.java:6)

C:\Program Files\Java\jdk-24\bin>
```

InputMismatchException: Input type doesn't match expected.

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner(System.in);
        int val=xyz.nextInt();
    }
}

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP
5.6
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:964)
    at java.base/java.util.Scanner.next(Scanner.java:1619)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2284)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2238)
    at VAPP.main(VAPP.java:6)
```

StackOverflowError: this error occur when you stack is full or when we have infinite recursion call

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        System.out.println("Good");
        main(x); //recursion
    }
}

Exception in thread "main" java.lang.StackOverflowError
    at java.base/java.nio.CharBuffer.wrap(CharBuffer.java:443)
    at java.base/sun.nio.cs.StreamEncoder.implWrite(StreamEncoder.java:266)
    at java.base/sun.nio.cs.StreamEncoder.write(StreamEncoder.java:122)
    at java.base/java.io.OutputStreamWriter.write(OutputStreamWriter.java:206)
```

OutOfMemoryError: when JVM runs out of memory or unable to allocate memory at program runtime then OutOfMemoryError occurs.

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        ArrayList<int[]> list= new ArrayList<int[]>();
        while(true) //infinite loop
        {
            list.add(new int[1000000]);
        }
    }
}

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at VAPP.main(VAPP.java:8)
```

UnsupportedOperationException: normally this exception occur when we try to modify the elements in collection or object and if there is any operation which is not supported by collection or object then JVM generate exception known as UnsupportedOperationException

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        // this method can create immutable collection means
        // we cannot modify collection as per our example list
        List<Integer> list=Arrays.asList(10,20);
        list.add(30); //generate UnsupportedOperationException at program runtime
    }
}

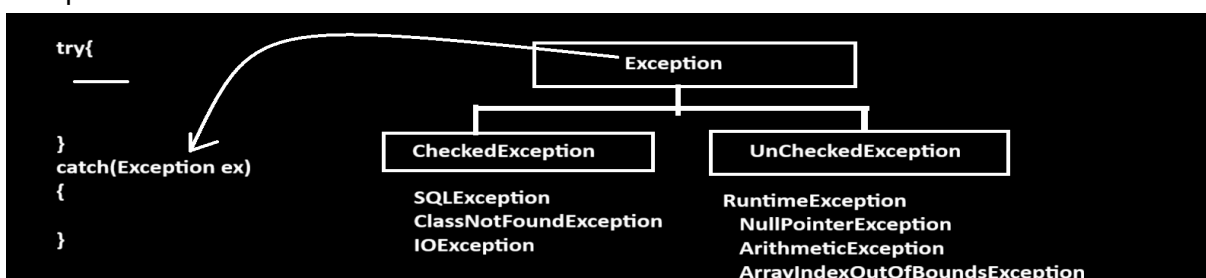
C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP
Exception in thread "main" java.lang.UnsupportedOperationException
    at java.base/java.util.AbstractList.add(AbstractList.java:155)
    at java.base/java.util.AbstractList.add(AbstractList.java:113)
    at VAPP.main(VAPP.java:8)

C:\Program Files\Java\jdk-24\bin>
```

How to handle more than one exception using a single catch block?

We can handle more than one exception using a single catch by passing the Exception class as a parameter in the catch block.




```

import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner(System.in);
        try{
            System.out.println("enter two values");
            int a=xyz.nextInt();//5.0
            int b=xyz.nextInt();
            int c=a/b;
            System.out.println("Division is "+c);
        }
        catch(Exception ex)
        { System.out.println("Error is "+ex);
        }
    }
}

```

```

Exception e = new ArithmeticException();
e = new InputMismatchException();

```

Finally block

Finally block is used for execute logic if exception generate in code or not
 Finally normally used for database connection ,file close etc purpose

How we can finally with catch block

```

try
{
    write here logic in which exception may be occur
}
finally
{
    write here your logics which want to execute always
}

```

Note: you can use finally directly with try

Q. Can we use try without catch?

Yes we can use try without catch by using finally block.

<pre> try { write here logic in which exception may be occur } finally { write here your logics which want to execute always } </pre>		<pre> try { } catch(exceptiontype ref) { } finally { write here your logics } </pre>
---	---	--

Note: when we use try with finally without catch and if exception in generated code then exception not handle just finally execute before exception but if we use try with finally using catch block then first exception get handle and after that finally get executed.

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner(System.in);
        try{
            System.out.println("enter two values");
            int a=xyz.nextInt();
            int b=xyz.nextInt();
            int c=a/b;
            System.out.println("Division is "+c);
        }
        finally{
            System.out.println("I can execute always");
        }
    }
}
```

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP

enter two values

9

0

I can execute always

Exception in thread "main" java.lang.ArithmeticException: / by zero

at VAPP.main(VAPP.java:10)

C:\Program Files\Java\jdk-24\bin>

Note: if we think about above code just finally get executed before exception means finally not handle exception just execute own block so if we want to handle the exception we have to use catch block.

So when we use try ,catch and finally then first catch execute handle exception then finally get execute

```
import java.util.*;
public class VAPP
{
    public static void main(String x[])
    {
        Scanner xyz = new Scanner(System.in);
        try{
            System.out.println("enter two values");
            int a=xyz.nextInt();
            int b=xyz.nextInt();
            int c=a/b;
            System.out.println("Division is "+c);
        }
        catch(Exception ex)
        {
            System.out.println("Error is "+ex);
        }
        finally{
            System.out.println("I can execute always");
        }
    }
}
```

C:\Program Files\Java\jdk-24\bin>javac VAPP.java

C:\Program Files\Java\jdk-24\bin>java VAPP

enter two values

10

0

Error is java.lang.ArithmeticException: / by zero

I can execute always

C:\Program Files\Java\jdk-24\bin>

Q. What is the difference between catch and finally?

1. Catch is block used for handle exception and finally is block execute logic every time Means finally not handle exception just execute own block
2. Catch execute only when exception generate in try block and finally execute every time if exception generate or not
3. Catch need parameter of exception type but finally not required parameter
4. Catch has higher execution priority than finally means when we use try ,catch and finally at same time then catch execute first before finally.

Q. What is the difference between finally,final, finalize?

Final : final is a keyword which we can use with variables, function and class.

1. Final variable means variable cannot modify its value
2. Final function means cannot override
3. Final class class means class cannot have child class

Finally: finally is blocked in exception handling which always executes if exceptions are generated in code or not. Finally need to try or we cannot use finally without try

finalize(): finalize() is a method of Object class which executes when we destroy objects manually and when we call [System.gc\(\)](#) method and this method is used for resource cleaning purposes.

Throws and throw clause

Throw clause : throw is a clause in exception handling which is used for handle the user define exceptions

Q. What user defines exceptions?

User defined exceptions means an exception defined by the user for its own use called as user defined exceptions or when a user create own exception class called as user define exceptions.

Q.Why do users need to define or create their own exception?

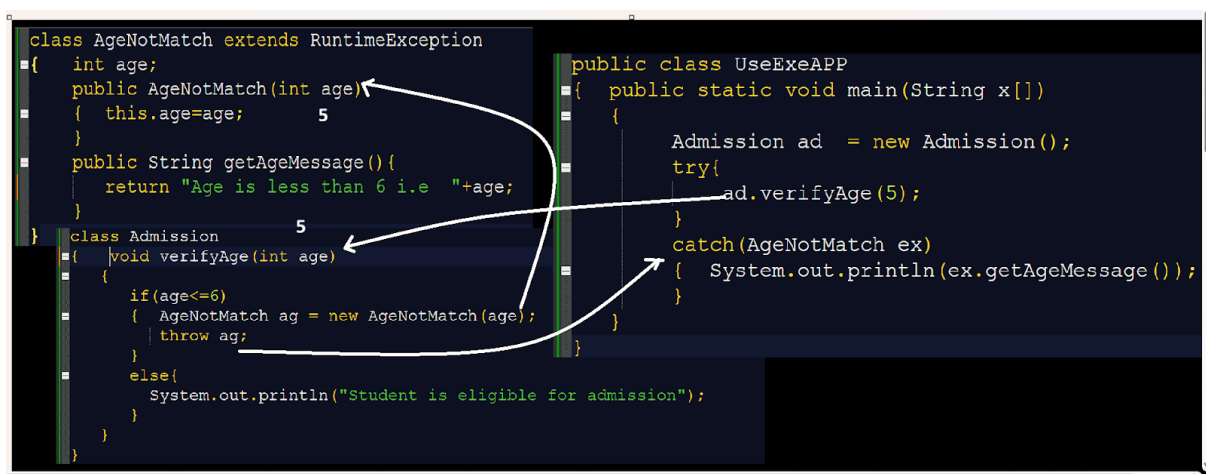
- When user has some logic which contain exceptions but java not provide class for handle that type of exception then user can create own exception called as user define exception
- When a user wants to generate an exception or error message according to project requirements then the user can create own exception class called as user defined exceptions.

How to create user define exception class

Example: Suppose we are working on School Admission and we have one clause: student age should be greater than equal 6 for 1st admission and if student age less than 6 then generate user defined exception and AgeNotMatch and return it.

Steps.

- Create user define class and inherit any exception class in it
- Create object of user exception class and throw it using throw keyword



```
class AgeNotMatch extends RuntimeException
{
    int age;
    public AgeNotMatch(int age){
        { this.age=age; 5
    }
    public String getAgeMessage(){
        return "Age is less than 6 i.e "+age;
    }
}
class Admission
{
    void verifyAge(int age)
    {
        if(age<=6)
        { AgeNotMatch ag = new AgeNotMatch(age);
        throw ag;
        }
        else{
            System.out.println("Student is eligible for admission");
        }
    }
}

public class UseExeAPP
{
    public static void main(String x[])
    {
        Admission ad = new Admission();
        try{
            ad.verifyAge(5);
        }
        catch (AgeNotMatch ex)
        { System.out.println(ex.getAgeMessage());
        }
    }
}
```

The screenshot displays two Java classes. The `AgeNotMatch` class extends `RuntimeException` and contains an `age` attribute, a constructor, and a `getAgeMessage()` method. The `Admission` class has a `verifyAge(int age)` method that checks if the age is less than or equal to 6; if so, it creates an `AgeNotMatch` object and throws it. The `UseExeAPP` class contains a `main` method that creates an `Admission` object and calls `verifyAge(5)`, which triggers the exception handling logic shown in the `catch` block.

Example:

```
class AgeNotMatch extends RuntimeException
{   int age;
    public AgeNotMatch(int age)
        {   this.age=age;
        }
    public String getAgeMessage(){
        return "Age is less than 6 i.e " +age;
    }
}
class Admission
{   void verifyAge(int age)
    {
        if(age<=6)
            {   AgeNotMatch ag = new AgeNotMatch(age);
                throw ag;
            }
        else{
            System.out.println("Student is eligible for admission");
        }
    }
}
public class UseExeAPP
{   public static void main(String x[])
    {
        Admission ad = new Admission();
        try{
            ad.verifyAge(5);
        }
        catch(AgeNotMatch ex)
        {   System.out.println(ex.getAgeMessage());
        }
    }
}
```

Example: WAP to create application for voting and generate InvalidVoter exception if voter age is less than 18 otherwise give message eligible for vote

```
class InvalidVoter extends RuntimeException
{   String getVoterMsg(){
    return "invalid voter";
    }
}
class VotingMachine
{   int age;
    void verifyVoter(int age)
        {   if(age<=18)
            {   InvalidVoter iv = new InvalidVoter();
            }
        }
}
```

```

        throw iv;
    }
    else{
        System.out.println("You are eligible for voting");
    }
}
}
}
public class IVAPP
{ public static void main(String x[])
{ try{
    VotingMachine im = new VotingMachine();
    im.verifyVoter(10);
}
catch(InvalidVoter iv)
{ System.out.println(iv.getVoterMsg());
}
}
}
}

```

throws : throws is a clause in exception handling which is used to handle the checked exceptions.

When we use throws then we are required to write throws clause with function definition and when we use throws clause with a function definition then not need to write try and catch in function definition block so throws clause can throw the exception or handle exception where function get executed.

```

class Div
{
    void setValue(int x,int y)throws ArithmeticException,ClassNotFoundException
    {
        System.out.println("Division is "+(x/y));
    }
}
public class DAPP
{
    public static void main(String x[])
    {
        try{
            Div d = new Div();
            d.setValue(10,0);
        }
        catch(Exception ex)
        { System.out.println("Error is "+ex);
        }
    }
}
}

```

Q. What is the difference between throw and throws?

throw	throws
Use for handle user defined exception	Use for handle checked exceptions
Need to create manual object of exception and throw it at function calling	Not need to create manual object of exception internally JVM create object and throw it
Can use with specified exception class	Can use with more than one exception class or generalised exception class

JDK 1.7 changes in exception in

1. **Try with resource bundle** : we can pass reference as parameter in try block

Syntax:

```
try(resource){  
  
}  
catch(exceptiontype ref)  
{  
}
```

2. Catch with multiple references using pipe operator

```
import java.util.*;  
public class DAPP  
{  
    public static void main(String x[])  
    {  
        try(Scanner xyz = new Scanner(System.in))  
        {  
            System.out.println("Enter two values");  
            int a=xyz.nextInt();  
            int b=xyz.nextInt();  
            int c=a/b;  
            System.out.println("division "+c);  
  
        }  
        catch(ArithmeticException | InputMismatchException |  
ArrayIndexOutOfBoundsException ex)  
        { System.out.println("Error is "+ex);  
        }  
    }  
}
```